

本节内容

操作系统的 运行机制

知识总览

操作系统的运行机制

两种指令

特权指令

非特权指令

两种处理器状态

核心态

用户态

两种程序

内核程序

应用程序

预备知识：程序是如何运行的？



C语言
代码

编译器
“翻译”

机器指令
(二进制)

一条高级语言的代码翻译过来可能会对多条机器指令

```
Int x = 1;  
x++;
```

```
100010101100001100011100  
001011000000001101001111  
100100100000001100000001  
001011000100111100000011
```



程序运行的过程其实就是CPU执行一条一条的机器指令的过程

基本命令

“小黑框”中使用的命令也注意与本节的“指令”区别

```
homlee — -bash — 59x12  
[zhanghonglindeMacBook-Air:Documents homlee$ cd 王道OS/  
zhanghonglindeMacBook-Air:王道OS homlee$ ls  
~$19年操作系统原题题库.docx      参考资料  
~$正文.doc                          知识点讲解  
习题讲解                            教材修改建议  
冲刺串讲  
zhanghonglindeMacBook-Air:王道OS homlee$ cd~  
[-bash: cd~: command not found  
zhanghonglindeMacBook-Air:王道OS homlee$ cd ~  
[zhanghonglindeMacBook-Air:~ homlee$ mkdir  
usage: mkdir [-pv] [-m mode] directory ...  
zhanghonglindeMacBook-Air:~ homlee$
```

预备知识：程序是如何运行的？



C语言
代码

编译器
“翻译”

机器指令
(二进制)

一条高级语言的代码翻译过来可能会对多条机器指令

```
Int x = 1;  
x++;
```

```
100010101100001100011100  
001011000000001101001111  
100100100000001100000001  
001011000100111100000011
```



程序运行的过程其实就是CPU执行一条一条的机器指令的过程

“指令”就是处理器（CPU）能识别、执行的最基本命令

注：很多人习惯把 Linux、Windows、MacOS 的“小黑框”中使用的命令也称为“指令”，其实这是“交互式命令接口”，注意与本节的“指令”区别开。本节中的“指令”指二进制机器指令

内核程序 v.s. 应用程序



C语言
代码

编译器
“翻译”

机器指令
(二进制)

一条高级语言的代码翻译过来可能会对多条机器指令

```
Int x = 1;  
x++;
```

```
100010101100001100011100  
001011000000001101001111  
100100100000001100000001  
001011000100111100000011
```



程序运行的过程其实就是CPU执行一条一条的机器指令的过程

我们普通程序员写的程序就是“应用程序”

微软、苹果有一帮人负责实现操作系统，他们写的是“内核程序”

由很多内核程序组成了“操作系统内核”，或简称“内核（Kernel）”

内核是操作系统最重要最核心的部分，也是最接近硬件的部分

甚至可以说，一个操作系统只要有内核就够了（eg: Docker—>仅需Linux内核）

操作系统的功能未必都在内核中，如图形化用户界面 GUI

特权指令 v.s. 非特权指令

```
Int x = 1;  
x++;
```



```
100010101100001100011100  
001011000000001101001111  
100100100000001100000001  
001011000100111100000011
```



应用程序只能使用“非特权指令”，如：
加法指令、减法指令等

程序运行的过程其实就是CPU执行一条一条的
机器指令的过程

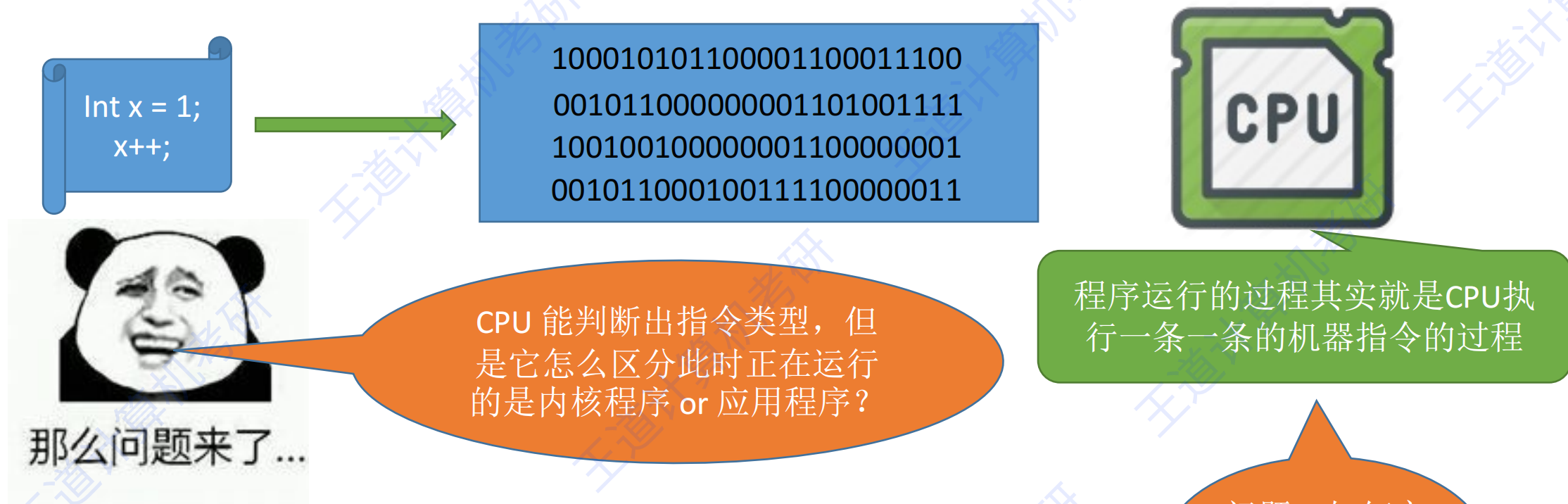
我们普通程序员写的程序就是“应用程序”

微软、苹果有一帮人负责实现操作系统，他们写的就是“内核程序”

操作系统内核作为“管理者”，有时会让CPU执行一些
“特权指令”，如：内存清零指令。这些指令影响重大，
只允许“管理者”——即操作系统内核来使用

在CPU设计和生产的时候就划分了特权指令和
非特权指令，因此CPU
执行一条指令前就能判
断出其类型

内核态 v.s. 用户态



CPU 有两种状态，“**内核态**”和“**用户态**”

处于**内核态**时，说明此时正在运行的是**内核程序**，此时可以执行**特权指令**

处于**用户态**时，说明此时正在运行的是**应用程序**，此时只能执行**非特权指令**

拓展：CPU 中有一个寄存器叫 **程序状态字寄存器（PSW）**，其中有个二进制位，1表示

“**内核态**”，0表示“**用户态**”

别名：内核态=核心态=**管态**；用户态=**目态**

问题：如何实现CPU状态的切换？



内核程序

应用程序

001011000000001101001111
100100100000001100000001
100010101100001100011100
.....

001011000000001101001111
100100100000001100000001
100010101100001100011100
001011000100111100000011

000001001011111101110101
100101010001011101110101
001010010010100010100010
.....

内核态

用户态



中断信号

操作系统内核在让出CPU之前，会用一条特权指令把PSW的标志位设置为“用户态”

处理中断信号的
内核程序

- ① 为“内核态”，
- ② 用户可以启动某个
- ③ 操作系统内核程序在合适的时候主动让出CPU，让该应用程序上CPU运行
- ④ 应用程序运行在“用户态”
- ⑤ 此时，一位猥琐黑客在应用程序中植入了一条特权指令，企图破坏系统...
- ⑥ CPU发现接下来要执行的这条指令是特权指令，但是自己又处于“用户态”
- ⑦ 这个非法事件会引发一个中断信号

CPU检测到中断信号后，会立即变为“核心态”，并停止运行当前的应用程序，转而运行处理中断信号的内核程序

- ⑧ “中断”使操作系统再次夺回CPU的控制权
- ⑨ 操作系统会对引发中断的事件进行处理，处理完了再把CPU使用权交给别的应用程序

内核态、用户态 的切换

内核态→用户态：执行一条**特权指令**——**修改PSW**的标志位为“用户态”，这个动作意味着操作系统将主动让出CPU使用权

用户态→内核态：由“**中断**”引发，**硬件自动完成变态过程**，触发中断信号意味着操作系统将强行夺回CPU的使用权

除了非法使用特权指令之外，还有很多事件会触发中断信号。一个共性是，**但凡需要操作系统介入的地方，都会触发中断信号**

一个故事：

- ① 刚开机时，CPU 为“**内核态**”，操作系统内核程序先上CPU运行
- ② 开机完成后，用户可以启动某个应用程序
- ③ 操作系统内核程序在合适的时候主动让出 CPU，让该应用程序上CPU运行
- ④ 应用程序运行在“用户态”
- ⑤ 此时，一位猥琐黑客在应用程序中植入了一条特权指令，企图破坏系统...
- ⑥ CPU发现接下来要执行的这条指令是特权指令，但是自己又处于“用户态”
- ⑦ 这个非法事件会引发一个**中断信号**

操作系统内核在让出CPU之前，会用一条**特权指令**把 **PSW** 的标志位设置为“**用户态**”

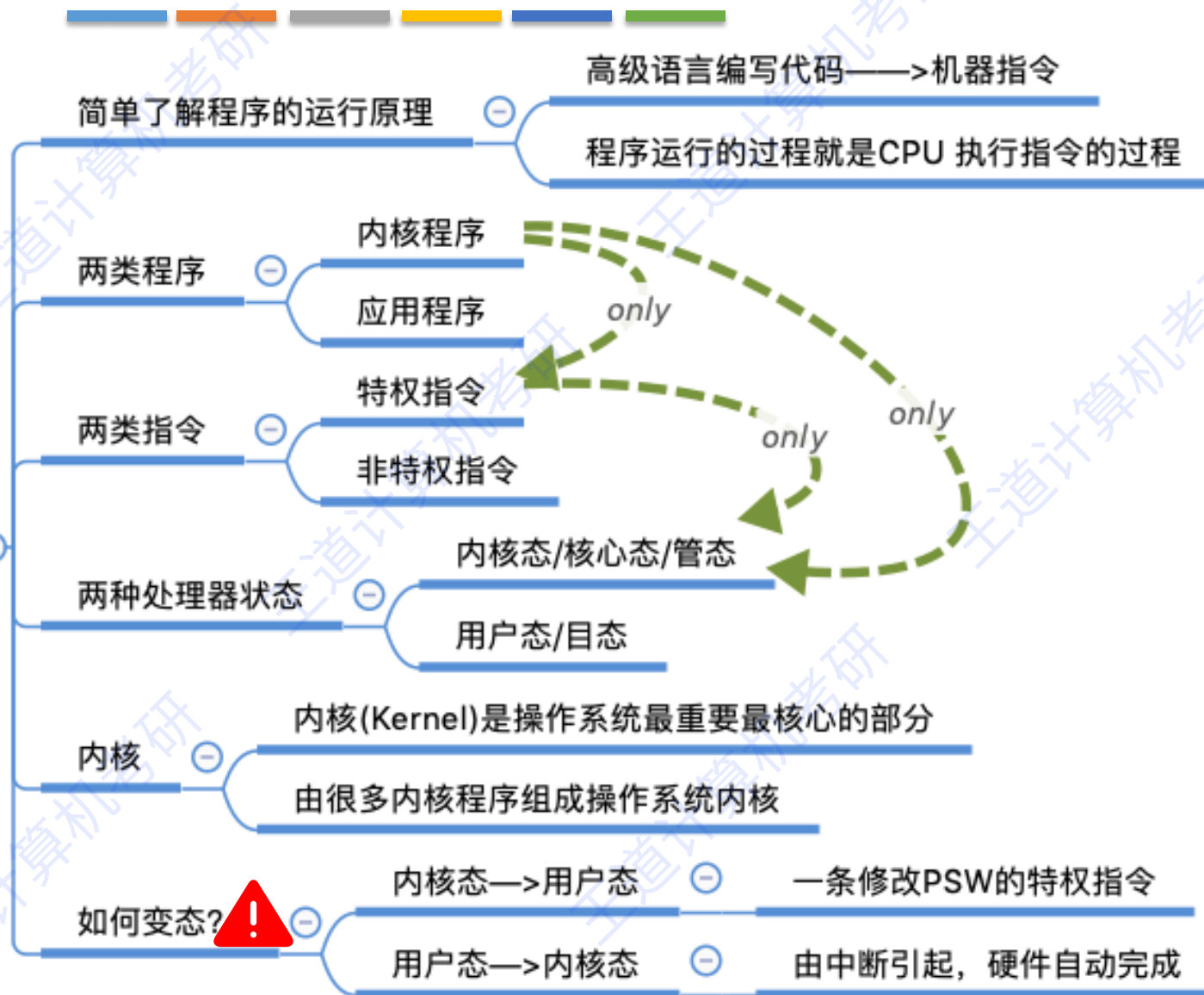
CPU检测到中断信号后，会立即变为“**核心态**”，并停止运行当前的应用程序，转而运行处理中断信号的内核程序

- ⑧ “中断”使操作系统再次夺回CPU的控制权
- ⑨ 操作系统会对引发中断的事件进行处理，处理完了再把CPU使用权交给别的应用程序

知识回顾与重要考点



操作系统的运行机制



Tips:

1. 都是高频考点, 很重要
2. 初学者不完全理解没关系, 放心大胆地往后学, 随着后面章节的学习, 理解会逐渐加深

本节内容

中断和异常

知识总览

中断和异常

中断的作用

中断的类型

内中断（也称“异常”）

外中断

中断机制的基本原理

中断的作用

“中断”会使CPU由用户态变为内核态，使操作系统重新夺回对CPU的控制权

CPU 上会运行两种程序，一种是**操作系统内核程序**，一种是**应用程序**

是整个系统的
管理者

在合适的情况下，操作系统内核会把CPU的使用权主动让给应用程序（第二章进程管理相关内容）

“中断”是**让操作系统内核夺回CPU使用权**的唯一途径

如果没有“中断”机制，那么一旦应用程序上CPU运行，CPU就会一直运行这个应用程序

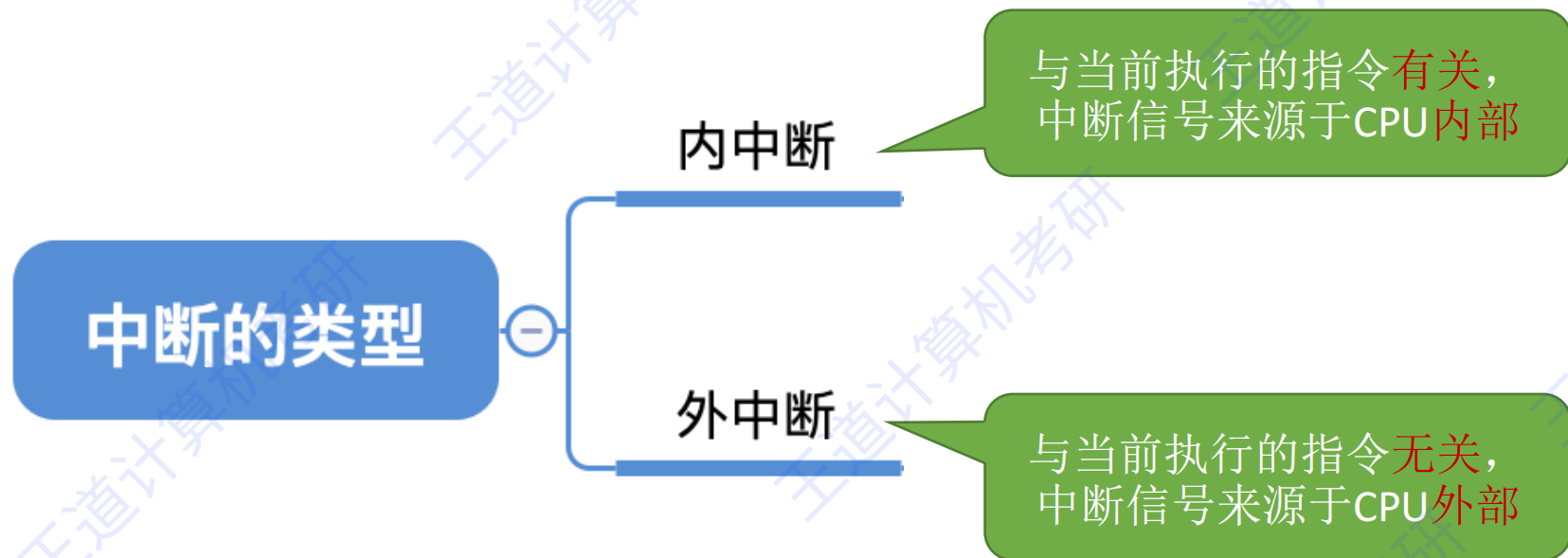


既如此，何来
“并发”！？

内核态→用户态：执行一条**特权指令**——**修改PSW**的标志位为“用户态”，这个动作意味着操作系统将主动让出CPU使用权

用户态→内核态：由“中断”引发，**硬件自动完成变态过程**，触发中断信号意味着操作系统将强行夺回CPU的使用权

中断的类型



与当前执行的指令有关，
中断信号来源于CPU内部

内中断的例子



应用程序



指令1
指令2
指令3 (特权指令)
指令4

处理中断
信号的
内核程序



指令1
指令2
指令3
.....

内核态

用户态



中断
信号

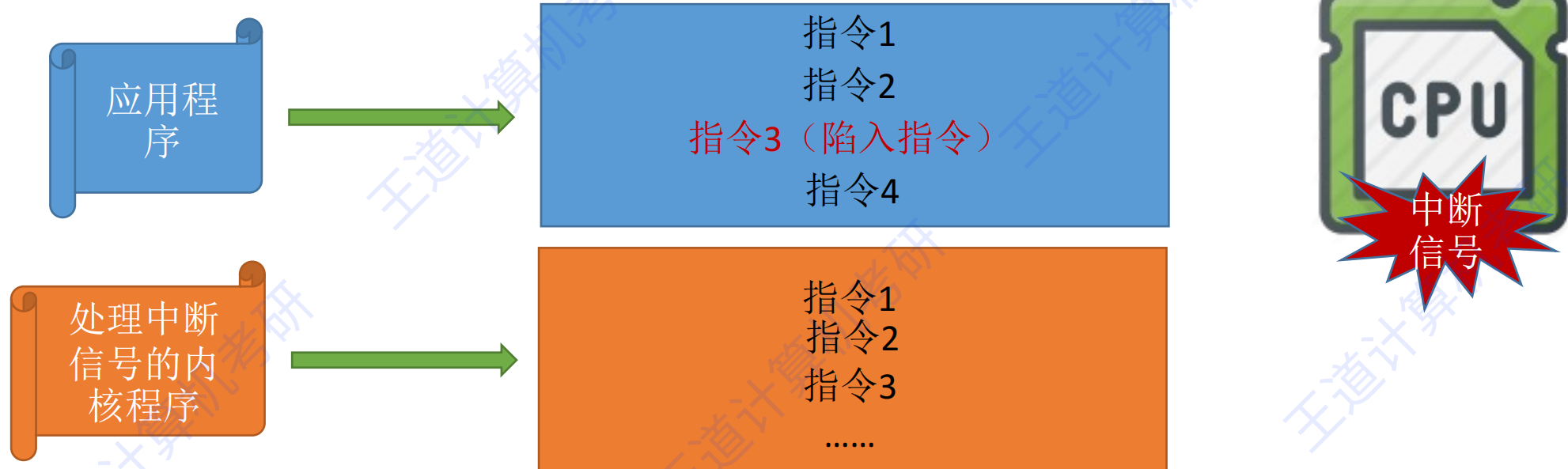


例子 1: 试图在用户态下执行特权指令
例子 2: 执行除法指令时发现除数为 0

若当前执行的指令是非法的，则
会引发一个中断信号

与当前执行的指令有关，
中断信号来源于CPU内部

内中断的例子



例子 3：有时候应用程序想请求操作系统内核的服务，此时会执行一条特殊的指令——陷入指令，该指令会引发一个内部中断信号

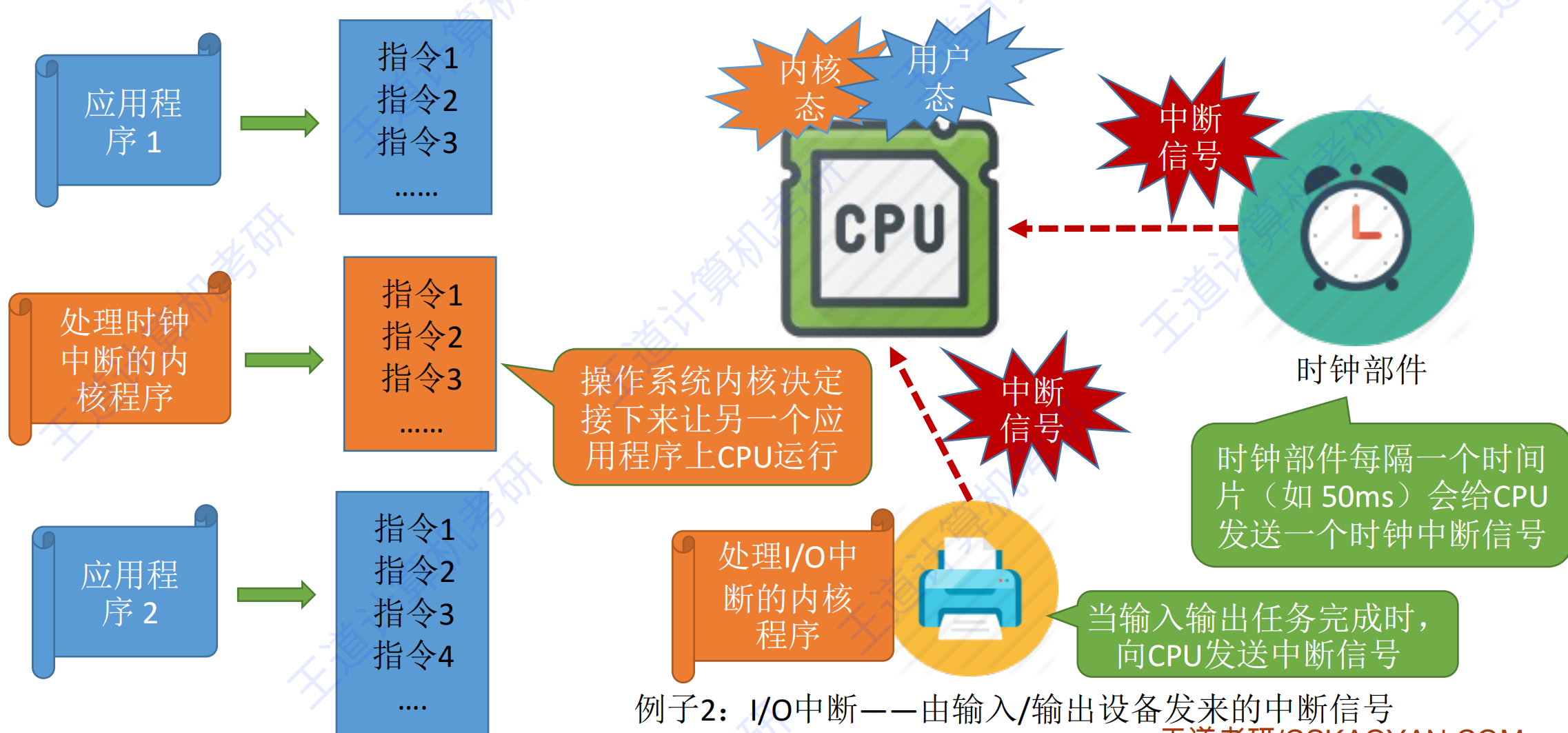
执行“陷入指令”，意味着应用程序主动地将CPU控制权还给操作系统内核。
“系统调用”就是通过陷入指令完成的

每一条指令执行结束时，CPU都会例行检查是否有外中断信号

外中断的例子

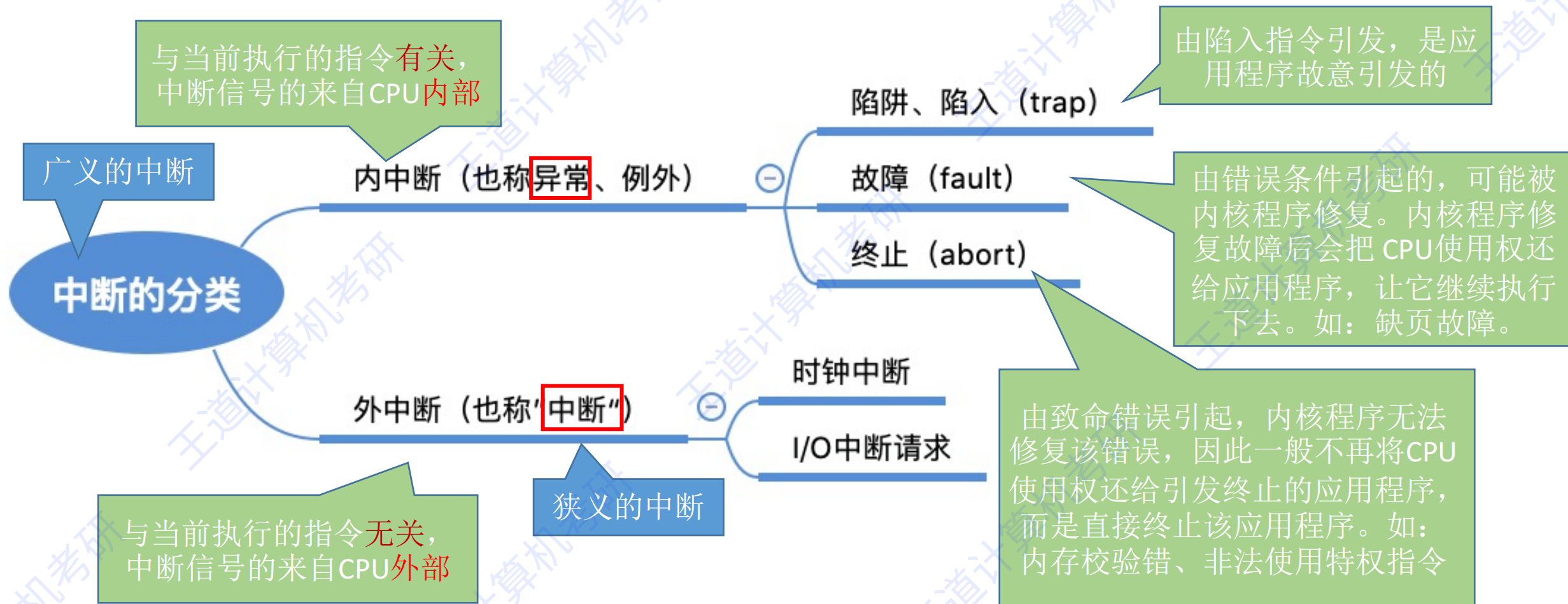
与当前执行的指令无关，中断信号来源于CPU外部

例子 1：时钟中断——由时钟部件发来的中断信号



例子2：I/O中断——由输入/输出设备发来的中断信号

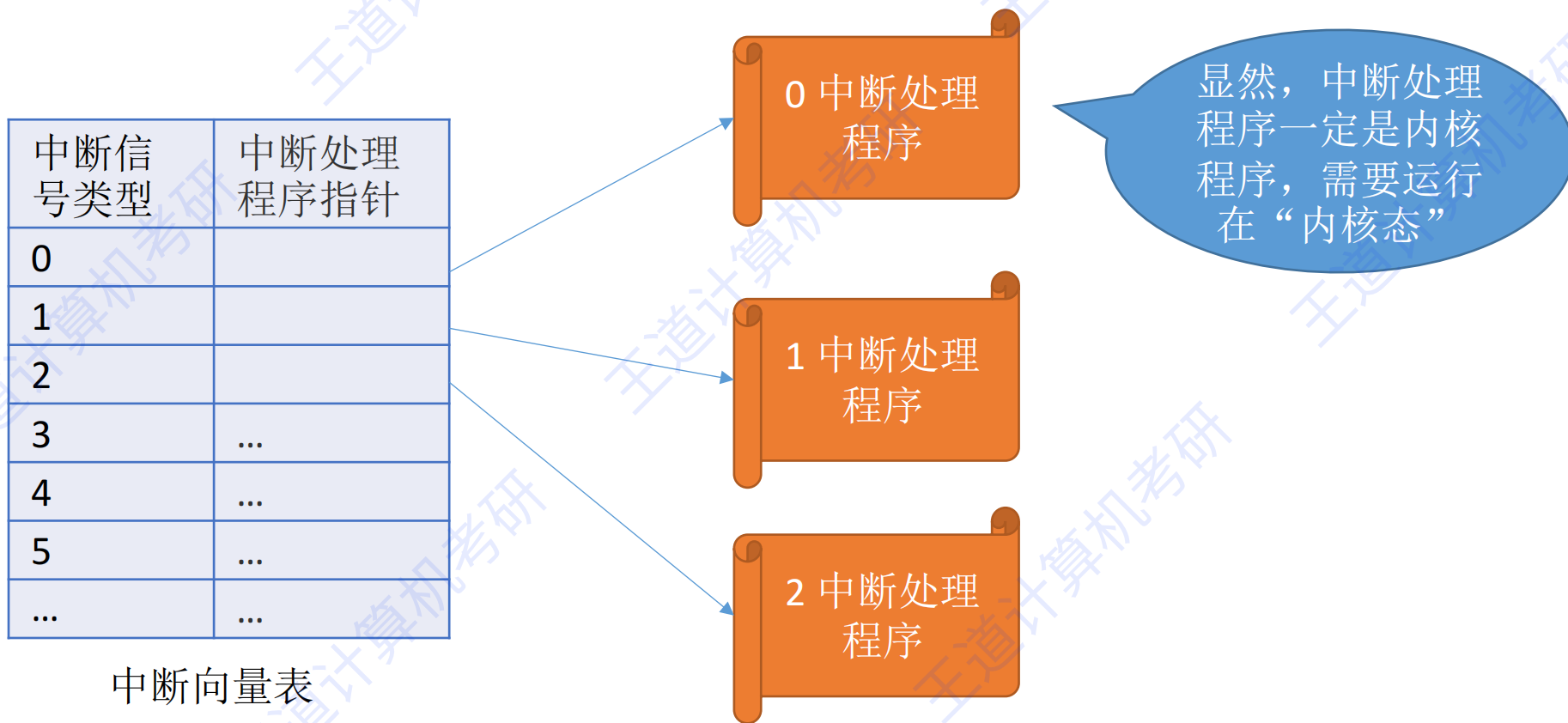
中断的分类



大多数的教材、试卷中，“中断”特指狭义的中断，即外中断。而内中断一般称为“异常”

中断机制的基本原理

不同的中断信号，需要用不同的中断处理程序来处理。当CPU检测到中断信号后，会根据中断信号的类型去查询“中断向量表”，以此来找到相应的中断处理程序在内存中的存放位置。



知识回顾与重要考点

没有中断机制，就不可能实现操作系统，不可能实现程序并发

让操作系统内核强行夺回CPU的控制权

中断的作用

使CPU从用户态变为内核态

中断的分类

内中断（也称异常、例外）

陷阱、陷入 (trap)

故障 (fault)

终止 (abort)

外中断（也称“中断”）

时钟中断

I/O中断请求

中断和异常

中断机制的基本实现原理

检查中断信号

内中断：CPU在执行指令时会检查是否有异常发生

外中断：每个指令周期末尾，CPU都会检查是否有外中断信号需要处理

找到相应的中断处理程序

通过“中断向量表”实现

本节内容

系统调用

知识总览

系统调用

什么是系统调用？

系统调用与库函数的区别

小例子：为什么系统调用是必须的？

什么功能要用系统调用实现？

系统调用的过程

什么是系统调用，有何作用？

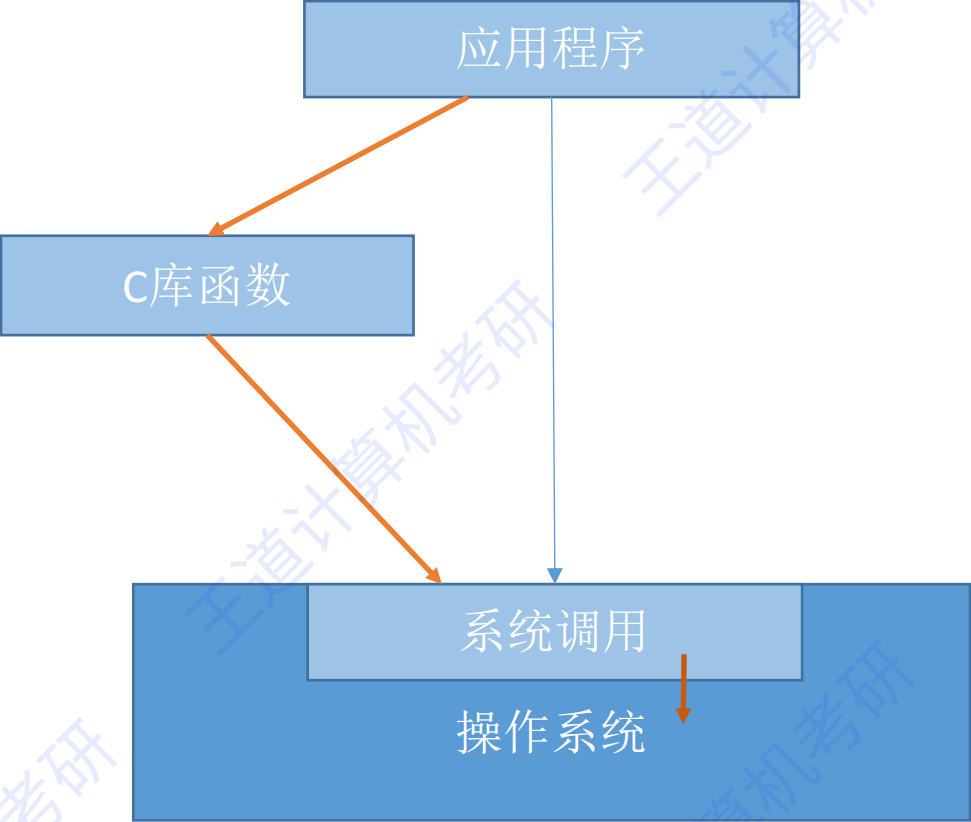
知识点回顾：

操作系统作为用户和计算机硬件之间的接口，需要向上提供一些简单易用的服务。主要包括命令接口和程序接口。其中，程序接口由一组**系统调用**组成。

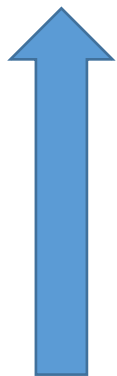


“系统调用”是操作系统提供给应用程序（程序员/编程人员）使用的接口，可以理解为一种可供应用程序调用的特殊函数，**应用程序可以通过系统调用来请求获得操作系统内核的服务**

系统调用与库函数的区别



普通应用程序	可直接进行系统调用，也可使用库函数。有的库函数涉及系统调用，有的不涉及
编程语言	向上提供库函数。有时会将系统调用封装成库函数，以隐藏系统调用的一些细节，使程序员编程更加方便。
操作系统	向上提供系统调用，使得上层程序能请求内核的服务
裸机	



不涉及系统调用的库函数：如的“取绝对值”的函数
涉及系统调用的库函数：如“创建一个新文件”的函数

小例子：为什么系统调用是必须的？



生活场景：去学校打印店打印论文，你按下了 WPS 的“打印”选项，打印机开始工作。

你的论文打印到一半时，另一位同学按下了 Word 的“打印”按钮，开始打印他自己的论文。

思考：如果两个进程可以随意地、并发地共享打印机资源，会发生什么情况？

两个进程并发运行，打印机设备交替地收到 WPS 和 Word 两个进程发来的打印请求，结果两篇论文的内容混杂在一起了...



解决方法：由操作系统内核对共享资源进行统一的管理，并向上提供“系统调用”，用户进程想要使用打印机这种共享资源，只能通过系统调用向操作系统内核发出请求。内核会对各个请求进行协调处理。



什么功能要用到系统调用？

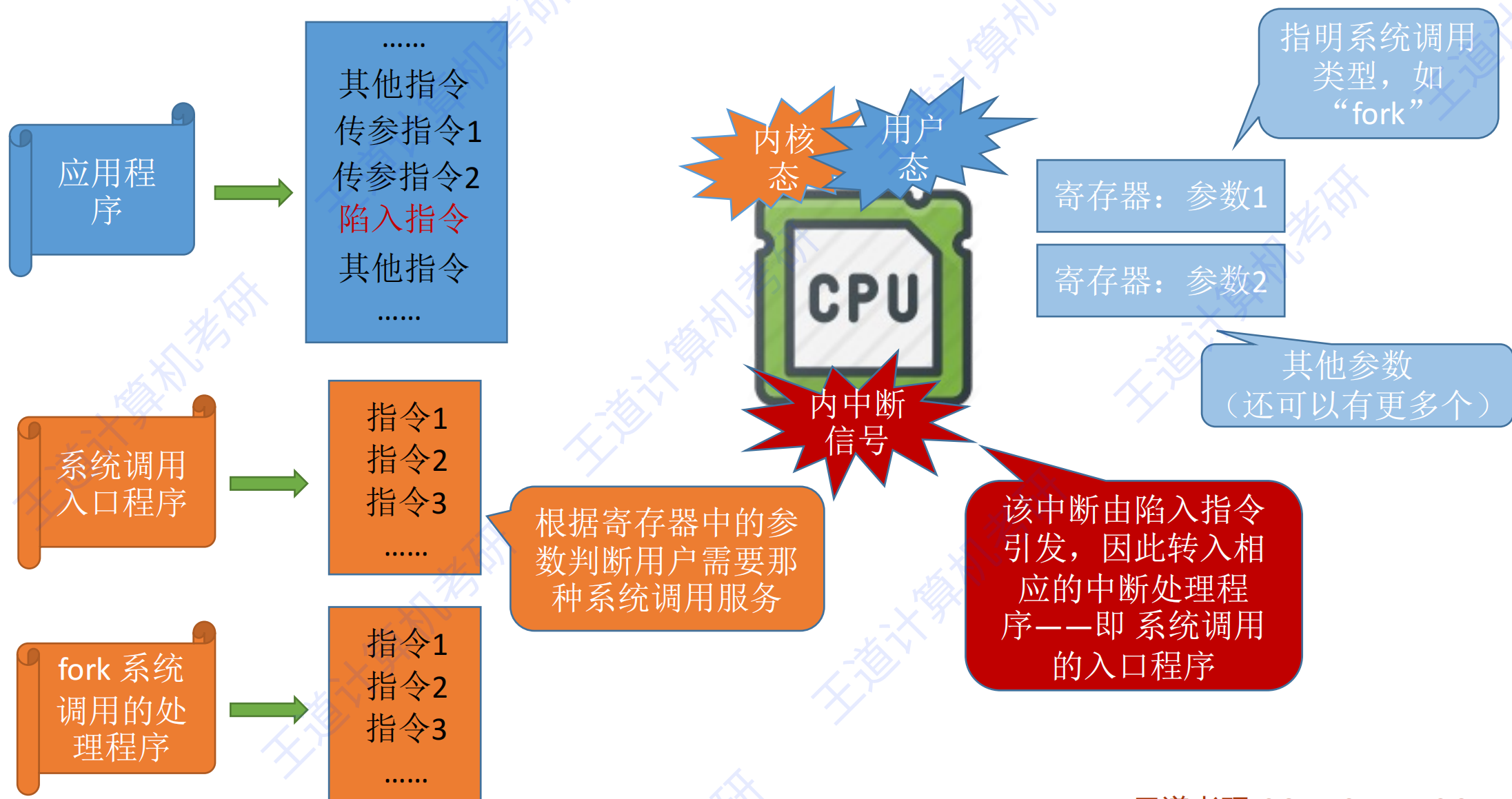


应用程序通过**系统调用**请求操作系统的服务。而系统中的各种共享资源都由操作系统内核统一掌管，因此**凡是与共享资源有关的操作**（如存储分配、**I/O操作**、文件管理等），都必须通过系统调用的方式向操作系统内核提出服务请求，由操作系统内核代为完成。这样可以**保证系统的稳定性和安全性**，防止用户进行非法操作。

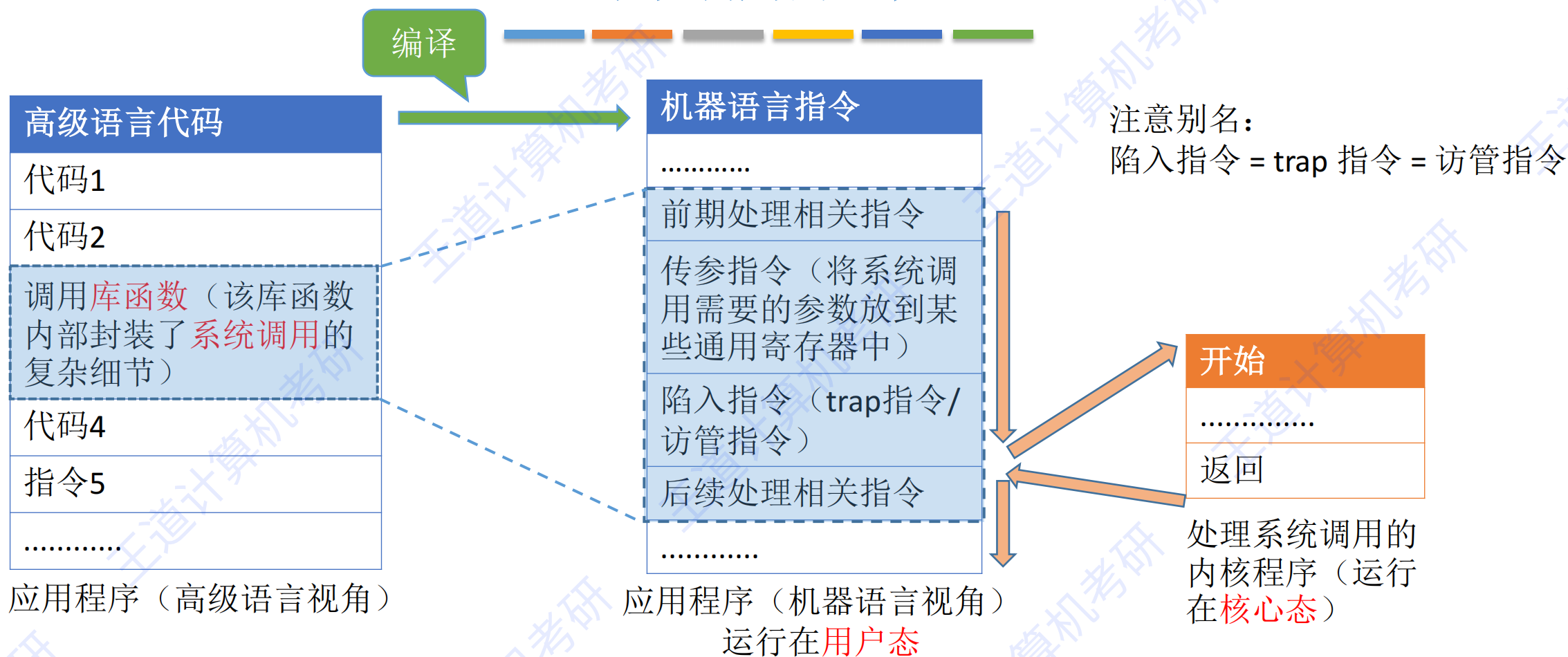


拓展：感兴趣的同学可以搜索“Linux 系统调用”，了解 Linux 操作系统提供了哪些系统调用

系统调用的过程



系统调用的过程



传递系统调用参数 → 执行陷入指令（用户态） → 执行相应的内核程序处理系统调用（核心态） → 返回应用程序

注意：1. 陷入指令是在用户态执行的，执行陷入指令之后立即引发一个内中断，使CPU进入核心态
2. 发出系统调用请求是在用户态，而对系统调用的相应处理在核心态下进行

知识回顾与重要考点

系统调用

什么是系统调用?

操作系统对应用程序/程序员提供的接口

系统调用与库函数的区别

有的库函数是对系统调用的进一步封装

有的库函数没有使用系统调用

小例子: 为什么系统调用是必须的?

Word 和 WPS 同时打印

什么功能要用系统调用实现?

设备管理

文件管理

进程控制

进程通信

内存管理

凡是与共享资源有关的操作、会直接影响到其他进程的操作,就一定需要操作系统介入,就需要通过系统调用来实现

系统调用的过程

传参

陷入指令/Trap/访管

由操作系统内核程序处理系统调用请求

返回应用程序

本节内容

番外篇

中国古代的操作系统

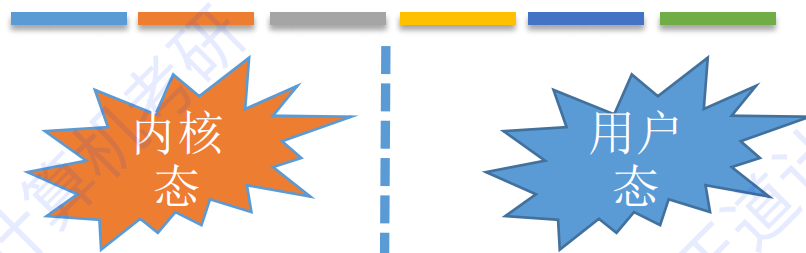
知识总览



喂，么么零吗？
有人在胡说八道！！

操作系统的设计思想早在古代的中国就被发明了

操作系统的运行机制



皇帝=管理者=
操作系统内核



小太监 = CPU
执行各种指令



大臣=普通应
用程序

特权指令和非特权指令



端茶

送水

拟一道圣旨

用户态下不可
执行特权指令



特权指令和非特权指令



内中断

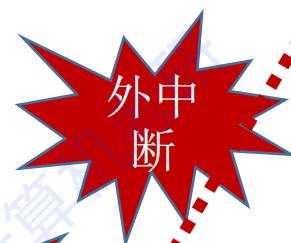


外中断



端茶

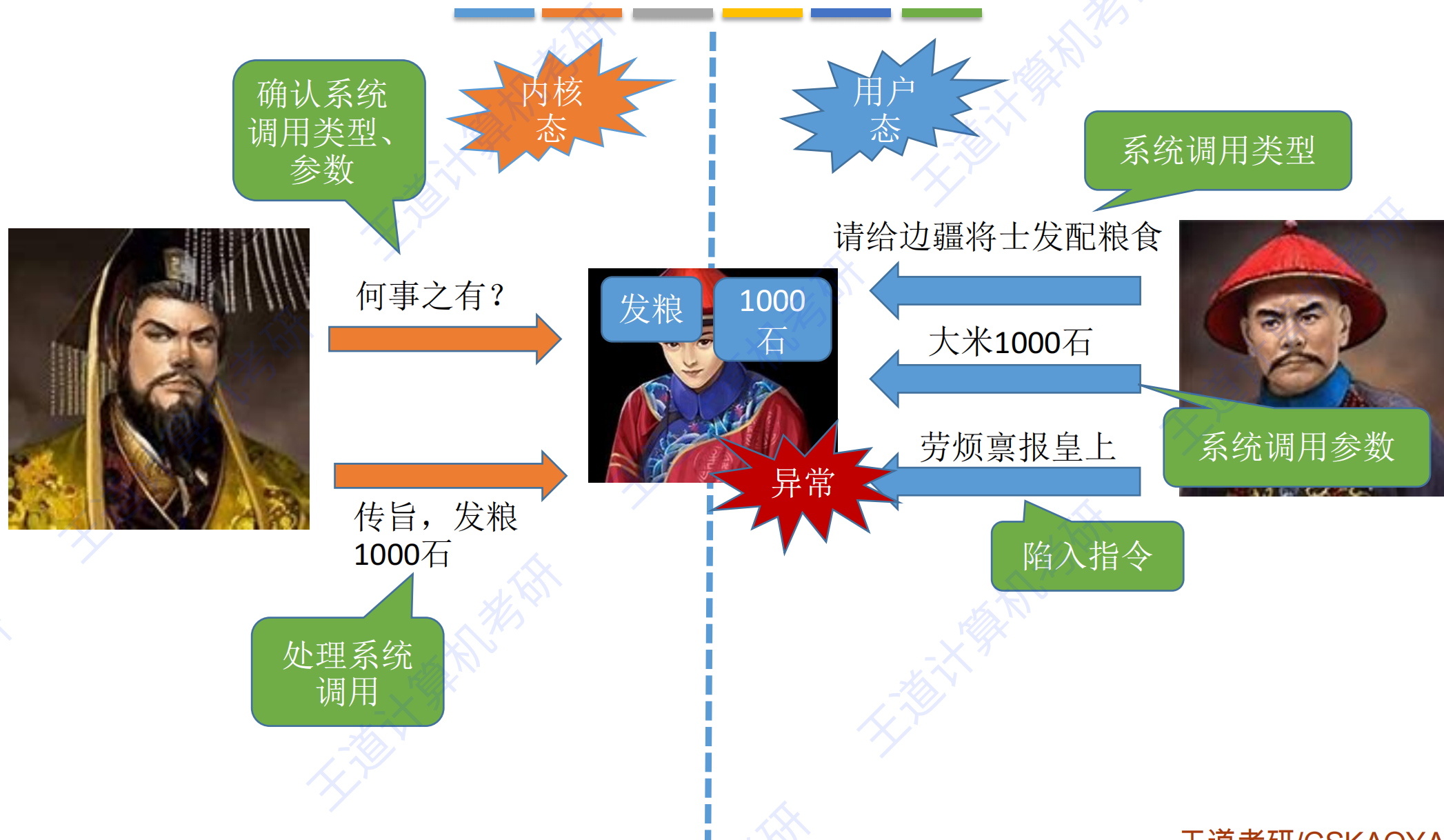
送水



检测到中断信号，
暂停运行应用程序，
转为内核态

传小李子觐见

系统调用



知识回顾与重要考点



仅供娱乐